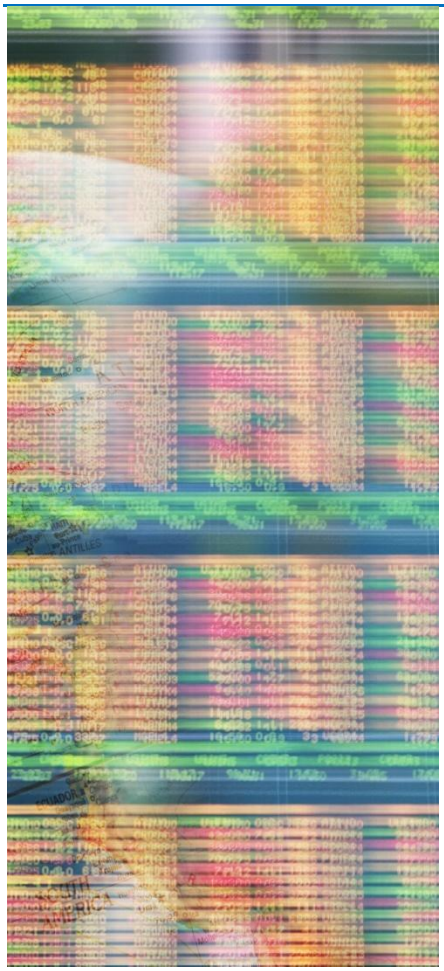
La guía de la optimación

Informática de Alto Rendimiento
Los Procesadores Escalables de Intel® Xeon®

La Guía para optimizar el WordPress* en la Plataforma de Procesadores Escalables Intel® Xeon® de 3ª Generación



Historial de Cambios 2

1. Introducción 3

2. La Arquitectura del WordPress* 4

3. El servidor físico o el servidor virtual (Bare-metal/On-prem) 5

3.1. Ajustes BIOS 5

3.2. Las Optimizaciones Sistema 5

3.2.1. Configuración de la Red 5

3.2.2. Configuración de la memoria RAM 5

3.2.3. Configuración de los Discos Duros 6

3.2.4. Configuración del CPI 6

3.2.5. Configuración del Kernel 6

3.3. Ajustes para la carga de trabajo WordPress 7

3.4. Optimizaciones para cifrado 8

3.5. Librerías de Software 8

4. La configuración de WordPress para Intel usando los paquetes de Bitnami 13

4.1. Configuración de la Azure 14

4.2. Configuración de la GCP 18

4.3. Configuración de la AWS 21

4.4. La pila de software 23

5. Configuración de evaluación comparativa 23

6. Prácticas recomendadas para pruebas y verificación 24

7. Más Referencias 24

Historial de Cambios

Date	Rev.	Descripción
04/06/2021	1.0	La primera versión
12/03/2021	2.0	Añadir las instrucciones para configurar el WordPress para Intel usando los paquetes de Bitnami

1. Introducción

Esta guía es para usuarios del WordPress*. Incluye las recomendaciones para configurar el BIOS, el OS y el WordPress con los ajustes que se puede aumentar el rendimiento en muchas situaciones. Tengamos en cuenta que confiamos en que los usuarios consideren cuidadosamente estas configuraciones porque los escenarios específicos del WordPress se pueden implementar de varias maneras. Los ajustes en esta guía dependen de WordPress que se ejecutan en un servidor de 2-sockets de Intel® con los procesadores escalables Intel® Xeon® de 3a Generación.

También esta guía tiene recomendaciones para configurar el WordPress en los servidores físicos (en el ingles: bare-metal) o configurar el WordPress para Intel usando los paquetes de Bitnami en las máquinas virtuales (la sigla en el inglés: VMs).

Si usa las imágenes de WordPress de Bitnami optimizadas por Intel en Azure/GCP/AWS, consulte la Sección 4. Suponemos que los usuarios están familiarizados con CSP (Cloud Security Platform o plataformas de seguridad de la nube) incluyendo la creación de VPC y la creación de máquinas virtuales y la configuración de reglas de firewall. Para obtener estas recomendaciones, usamos el Ubuntu* 20.04.1 LTS y el WordPress v5.2. Se tiene que descargar e instalar el OS y otro software antes de instalar las cargas de trabajo porque los binarios de software dependientes o licencias de software dependientes no se distribuyen con las cargas de trabajo.

El WordPress es una plataforma para páginas webs y un sistema de gestión de contenidos (la sigla en el inglés: CMS). Se puede aprender más en: https://wordpress.org. Esta carga de trabajo usa WordPress para publicar contenidos. El WordPress usa el servidor web, Nginx, con los servidores, PHP-FPM, la base de datos, MariaDB, y el cliente, Siege. Estos pasos para configurar se van a explicar en las secciones subsecuentes. Estos pueden estar implementado en los servidores físicos o los servidores virtuales que están en sus centros de datos o en la nube.

Los procesadores escalables Intel® Xeon® de 3ª Generación contienen plataformas que permiten optimización de las cargas de trabajo con aceleración de la IA (Inteligencia Artificial) incorporada. Estos procesadores tienen el rendimiento que ayudan acelerar el impacto transformador de los datos desde el perímetro (Edge) a la nube (Cloud). Los mejoramientos específicos de WordPress incluyen:

- El rendimiento mejorado
- Más Intel® Ultra Path Interconnect
- Intel® Advanced Vector Extensions
- Aumenta Intel® Deep Learning Boost with VNNI
- Aumenta DDR4 Memory Speed & Capacity
- Intel® Security Essentials e Intel® Security Libraries for Data Center
- Intel® Speed Select Technology
- Apoyar para las series: Intel® Optane™ Persistent Memory 200

2. La Arquitectura del WordPress*

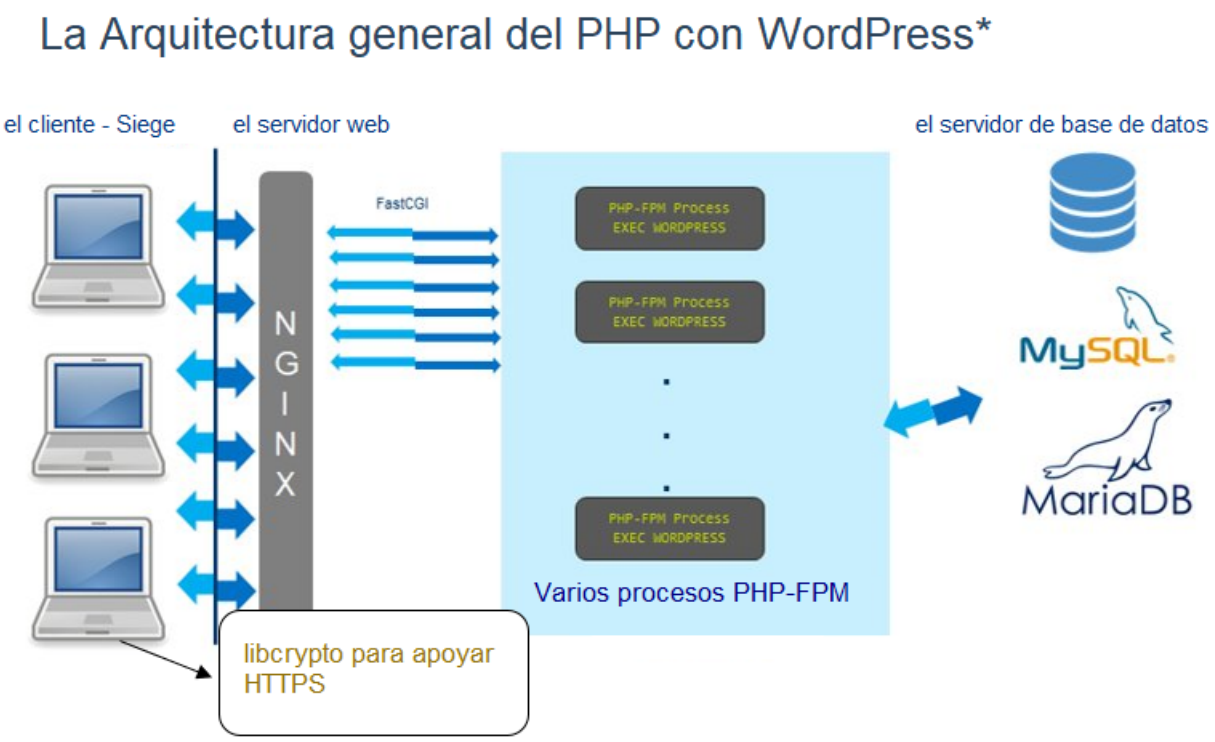


*Ilustración 1: La Arquitectura general del PHP con WordPress**

Como se muestra en la Ilustración 1, la carga de trabajo WordPress es una carga de trabajo con tres componentes críticos: el servidor, la base de datos y el cliente. Esta carga de trabajo empieza el Nginx como el servidor "proxy", iniciando el PHP y conectando la base de datos, MariaDB. El cliente, Siege, ejecuta las peticiones de la web e informa el flujo de las peticiones de la web como los parámetros primarios.

La carga de trabajo WordPress se puede ejecutar por único nodo o por muchos nodos:

- En un único nodo todos los tres componentes están en un sistema que está probando (en el inglés: System Under Test o SUT)

- En el multi-nodo los tres componentes están en sistemas diferentes pero el PHP y el Nginx están en un SUT. Configure el cliente y la base de datos para evitar tráfico de red. Entonces el SUT procesará solamente peticiones que el cliente o la base de datos pueda manejar efectivamente y no tendría un impacto negativo en el rendimiento.

En este sistema, el WordPress usa HTTPS "crypto support" para apoyar la connecion privada entre el Siege y el Nginx.

3. El servidor físico o el servidor virtual (Bare-metal/On-prem)

3.1. Ajustes BIOS

Regrese a la configuración base del BIOS. Siga estas recomendaciones:

Ajustes	Recomendaciones
Hyper-Threading	Activar
Hardware Prefetcher	Activar
L2 RFO Prefetch Disable	Desactivar
Adjacent Cache Prefetch	Activar
DCU Streamer Prefetcher	Activar
DCU IP Prefetcher	Activar
LLC Prefetch	Activar
Total Memory Encryption (TME)	Desactivar
SNC (Sub NUMA)	Desactivar
UMA-Based Clustering	Hemisphere (2-clusters)
Snoop mode for AD	Desactivar
Snoopy mode for 2LM	Desactivar
Boot performance mode	Max Performance
Turbo Mode	Activar
Hardware P-State	Native Mode
Local/Remote Threshold	Auto

3.2. Las Optimizaciones Sistema

3.2.1. Configuración de la Red

Cuando el SUT, la base de datos y el cliente son ejecutados en tres servidores diferentes, los tres servidores deben estar conectados con, al menos, una tarjeta 10G NIC en la misma subred privada. Es importante que verifique el rendimiento de la red usando herramientas como sar/nload, antes de ejecutar la carga de trabajo WordPress.

3.2.2. Configuración de la memoria RAM

Por lo menos una canal de la memoria DIMM debe ser poblada si no, la competición en el cache de la base de datos podría causar un problema porque la utilización del CPU no es óptima.

3.2.3. Configuración de los Discos Duros

<No hay instrucciones especiales para discos duros>

3.2.4. Configuración del CPU

Esta guía ha desarrollado para optimizar los procesadores escalables Intel® Xeon® de 3ª generación. Se ajuste el regulador de frecuencia de todos los núcleos a “performance”:

```
echo "performance" | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
```

3.2.5. Configuración del Kernel

Estos ajustes del kernel solamente son las recomendaciones para el WordPress. Se vacia los búferes del sistema de archivos usando este mandato:

```
sudo sync
```

Se vacia las memorias caché incluyendo pagecache, dentries y inodes, usando este mandato. La referencia: https://stackoverflow.com/questions/29870068/what-are-pagecache-dentries-inodes

```
sudo sh -c 'echo 3 >/proc/sys/vm/drop_caches'
```

Se vacia la memoria “swap” usando este mandato:

```
sudo swapoff -a > /dev/null 2>&1
sudo swapon -a > /dev/null 2>&1
```

Si lo instalado, se desactiva el auditd daemon usando este mandato:

```
sudo service auditd stop || true
```

Se activa ip_conntrack usando este mandato:

```
sudo modprobe ip_conntrack
```

Se aumenta el tamaño de la hashtable nf_conntrack depende de la memoria RAM disponible. Ajusta el tamaño lo que se usa hacer un seguimiento de las conexiones. Revisa la kern.log. Si ve mensajes como “nf_conntrack: table full, dropped packet”, entonces se debe aumentar el número de los conexiones que están siendo seguimiento.

```
echo 512000 | sudo tee /proc/sys/net/netfilter/nf_conntrack_max
```

Se permite la reutilización de los sockets TCP en el estado de TIME_WAIT cuando hay conexiones nuevas.

```
echo 1 | sudo tee /proc/sys/net/ipv4/tcp_tw_reuse
```

Para optimizar el número de los archivos abiertos, se usa la cuota de espacio regular o se usa la cuota de espacio del usuario. También, se usa una de estas cuotas para optimizar los espacios de las direcciones de memoria que están bloqueado. En la carpeta, /etc/security/limits.conf, se ajusta los ajustes siguientes:

```
* soft    nofile  1000000
* hard    nofile  1000000
* soft    memlock unlimited
* hard    memlock unlimited
```

3.3. Ajustes para la carga de trabajo WordPress

Para obtener el rendimiento óptimo, se conecta cada php-fpm procesa a uno CPU virtual (vCPU).

Si el WordPress es ejecutado en uno socket, se activa MySQL QueryCache con nuestros tamaños preestablecidos de caché:

```
open_files_limit=4000
query_cache_limit=128K
query_cache_size=1M
query_cache_type=1
```

Si el WordPress es ejecutado en dos sockets, se desactiva QueryCache. Se podría mejor desactivar query_cache completamente en los bases de datos grande. También se podría mejor desactivar los servidores que tienen usualmente >= 28C. El ajuste, query_cache 1S, es una recomendación cuando la memoria disponible es mucho más pequeña.

Se ajusta el número de procesas servidores basado en el número de CPUs que incluido núcleos lógicos. Le recomendamos que si hay menos o igual que 16, la cuenta de los trabajadores servidores debe estar 2.5 * la cuenta de CPUs. Si la cuenta de CPUs es menos o igual que 95, la cuenta de los trabajadores servidores es 2 * la cuenta de CPUs. De lo contrario la cuenta es 1.5 * la cuenta de CPUs.

Para ajustar el número de procesas clientes, mira a los procesas clientes y elija aquellos con el rendimiento mejor. Haga ajustes si necesario.

Para reducir el acceso de un nodo numa a otro y aumentar el rendimiento, los instancias WordPress diferentes pueden usar contenedores diferentes para hacer enlazados a sockets diferentes.

3.4. Optimizaciones para cifrado

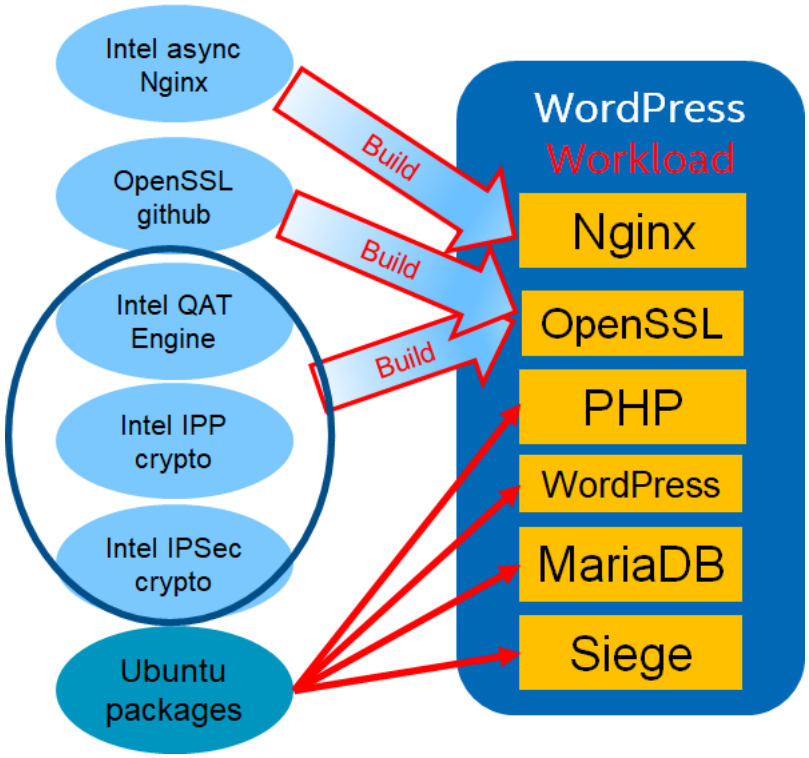


Ilustración 2: La pila de software WordPress que le recomendamos

El Hypertext Transfer Protocol Secure (HTTPS) se usa para comunicación segura por la red. Hay librerías especiales que soporta comunicación cifrada de WordPress. Ilustración 2 se muestra estas librerías.

3.5. Librerías Software

El apunte: versiones de las librerías pueden cambiar a lo largo del tiempo. Elija la versión actual.

Libraries	**Versión**	**Enlace**
OpenSSL	OpenSSL 1.1.1f	https://github.com/openssl/openssl
IPP-Crypto	ippcp_2020u3	https://github.com/intel/ipp-crypto
QAT engine	v0.6.5	https://github.com/intel/QAT_Engine
Async Nginx	v0.4.5	https://github.com/intel/asynch_mode_nginx
Multi-buffer crypto for IPSec	V0.55	https://github.com/intel/intel-ipsec-mb

Cifrados utilizados:

ECDHE-RSA-AES128-GCM-SHA256

TLS_AES_256_GCM_SHA384
Por ejemplo,

- TLSv1.2 - ECDHE-RSA-AES128-GCM-SHA256
- TLSv1.3 - TLS_AES_256_GCM_SHA384

Instalar librerías software:

Se necesitan estos paquetes:

```
sudo apt-get install libudev-dev libssl-dev libboost-all-dev zlib1g-dev autoconf make build-essential libnl-3-dev libnl-genl-3-dev yasm cmake libpcre3 libpcre3-dev
```

Las variables de entorno:

#ROOT_DIR es la carpeta primaria cuando git repos están siendo descarga

```
export OPENSSL_SOURCE=$ROOT_DIR/openssl/
export OPENSSL_LIB=$ROOT_DIR/openssl_install/
export OPENSSL_ENGINES=$ROOT_DIR/openssl_install/lib/engines-1.1/
export NGINX_INSTALL_DIR=$ROOT_DIR/nginx_install/
export IFMA_SOURCE=$ROOT_DIR/ipp-crypto/sources/ippcp/crypto_mb/
export IPSEC_SOURCE=$ROOT_DIR/$SSL_DIR/intel-ipsec-mb/
export IPSEC_INSTALL=$ROOT_DIR/$SSL_DIR/intel-ipsec-mb-install/
```

OpenSSL:

```
git clone https://github.com/openssl/openssl.git
cd openssl
git checkout OpenSSL_1_1_1f
make clean
make distclean
./config --prefix=$OPENSSL_LIB -Wl,-rpath,$OPENSSL_SOURCE
make update
make depend
make -j
make install
```

IPP-Crypto

```
git clone  https://github.com/intel/ipp-crypto.git
cd ipp-crypto
git checkout ipp-crypto_2020_update3
cd sources/ippcp/crypto_mb
cmake . -B"./build" -DOPENSSL_INCLUDE_DIR=$OPENSSL_SOURCE/include/ -DOPENSSL_LIBRARIES=$OPENSSL_LIB -DOPENSSL_ROOT_DIR=$OPENSSL_SOURCE/
cd build
make -j
ls $IFMA_SOURCE/build/bin/libcrypto_mb.so #File must exist
mkdir $IFMA_SOURCE/lib/
ln -s $IFMA_SOURCE/build/bin/libcrypto_mb.so $IFMA_SOURCE/lib/
```

IPSec library

```
cd ROOT_SOURCE
git clone https://github.com/intel/intel-ipsec-mb.git
cd intel-ipsec-mb
git checkout v0.55
make
```

```
make install PREFIX=$IPSEC_INSTALL
```

QAT Engine:

```
cp $IPSEC_INSTALL/lib/* $IFMA_SOURCE/lib/
cp $IPSEC_INSTALL/include/* $IFMA_SOURCE/include/
```

```
git clone https://github.com/intel/QAT_Engine.git
cd QAT_Engine
git checkout v0.6.5
for i in 1 2;
do
  multi_buffer_string="--enable-qat_sw --with-qat_sw_install_dir=$IFMA_SOURCE --with-
openssl_install_dir=$OPENSSL_LIB"

  make clean
  ./autogen.sh
  ./configure $multi_buffer_string
  make -j
  make install -j
done
```

```
ls $OPENSSL_LIB/lib/engines-1.1
afalg.so  capi.so  padlock.so  qatengine.la  qatengine.so
```

Async Nginx:

```
git clone https://github.com/intel/asynch_mode_nginx.git
cd asynch_mode_nginx
git checkout v0.4.5
./configure \
      --prefix=$NGINX_INSTALL_DIR \
      --with-http_ssl_module \
      --add-dynamic-module=modules/nginx_qat_module \
      --with-cc-opt="-DNGX_SECURE_MEM -I$OPENSSL_LIB/include -Wno-error=deprecated-declarations" \
     --with-ld-opt="-Wl,-rpath=$OPENSSL_LIB/lib -L$OPENSSL_LIB/lib"
make
make install
```

Archivos librerías que se usa con WordPress:

Copia los archivos a una carpeta llamada qatengine. Guarda la ruta en esta variable de entorno: $qatengine

```
cd $qatengine
mkdir lib bin engine ngx_modules
cp $OPENSSL_LIB/bin/openssl $qatengine/bin
cp $NGINX_INSTALL_DIR/sbin/nginx $qatengine/bin
cp $OPENSSL_LIB/lib/engines-1.1/qatengine.so $qatengine/engine
cp $OPENSSL_LIB/lib/libcrypto.so.1.1 $qatengine/lib
cp $OPENSSL_LIB/lib/libssl.so.1.1 $qatengine/lib
cp $IFMA_SOURCE/lib/libcrypto_mb.so $qatengine/lib
cp $IPSEC_INSTALL/lib/libIPSec_MB.so.0 $qatengine/lib
cp $NGINX_INSTALL_DIR/modules/ngx_ssl_engine_qat_module.so $qatengine/ngx_modules
cp $OPENSSL_LIB/ssl/openssl.cnf $qatengine
```

```
export LD_LIBRARY_PATH=$qatengine/lib
export OPENSSL_ENGINES=$qatengine/engine
export PATH=$qatengine/bin:$PATH
export OPENSSL_CONF=$qatengine/openssl.cnf
#To check if qatengine can be loaded by openssl
openssl speed -engine qatengine -elapsed -async_jobs 8 rsa2048
```

Generación de llaves:

Se usa estos comandos generar la llave/certificado de RSA (en el inglés: RSA key/certficate):

```
openssl req -x509 -sha256 -nodes -days 365 -newkey rsa:2048 -keyout certs/server.key -out certs/server.crt -subj "/C=/ST=/L=/O=/OU=/CN=$(hostname)"
Generate key/certficate for DSA:
openssl ecparam -genkey -out certs/key.pem -name prime256v1
openssl req -x509 -new -key certs/key.pem -out certs/cert.pem -subj "/C=/ST=/L=/O=/OU=/CN=$(hostname)"
```

El archivo configuración de Nginx:

```
worker_processes 128; #Ajuste al número de núcleos lógicos
load_module ngx_modules/ngx_ssl_engine_qat_module.so;
ssl_engine{
  use_engine qatengine;
  default_algorithms ALL;
  qat_engine{
    qat_notify_mode poll;
    qat_offload_mode async;
    qat_external_poll_interval 1;
}}
worker_rlimit_nofile 1000000;
events
{
 use epoll;
 worker_connections 8192;
 multi_accept on;
 accept_mutex on;
}
http {
 ssl_buffer_size 65536;
 types_hash_max_size 4096;
 types_hash_bucket_size 64;
 sendfile       on;

server {
   listen [::]:__HTTP_PORT__ reuseport backlog=131072 so_keepalive=off rcvbuf=65536 sndbuf=65536 default_server;
   listen __HTTP_PORT__ reuseport backlog=131072 so_keepalive=off rcvbuf=65536 sndbuf=65536 default_server;
   sendfile on;
   keepalive_timeout 0s;
   tcp_nopush on;
   tcp_nodelay on;
   ssl_verify_client off;
   ssl_session_tickets off;
   lingering_close off;
   lingering_time 1;
   ssl on;
```

```
    ssl_asynch on;

    ssl_certificate  server.crt # Para DSA use cert.pem en vez de server.crt
    ssl_certificate_key server.key # Para DSA use key.pem en vez de server.key

    ssl_session_timeout  300s;
    ssl_protocols  TLSv1.2;
    ssl_prefer_server_ciphers   on;
    ssl_ciphers ECDHE-RSA-AES128-GCM-SHA256;
    ssl_ecdh_curve secp384r1; // se recomienda usar secp384r1 con la curva que se usa, por ejemplo, secp384r1/
prime256v1
    ...
}}
```

El apunte: Tiene que considerar el tipo de cifrado y la curva que se usa para cambiar las llaves. También, tiene que considerar la método de autenticación. Los dos van a cambiar el rendimiento. Depende de la curva y cifrado que se usa, el rendimiento de su carga de trabajo podría estar diferente. Le recomendamos que tiene en cuenta esto cuando ajuste su carga de trabajo.

La referencia: Building Software Acceleration Features in the Intel® Quick Assist Technology (Intel® QAT) Engine for OpenSSL* 1.1.1

4. La configuración de WordPress para Intel usando los paquetes de Bitnami

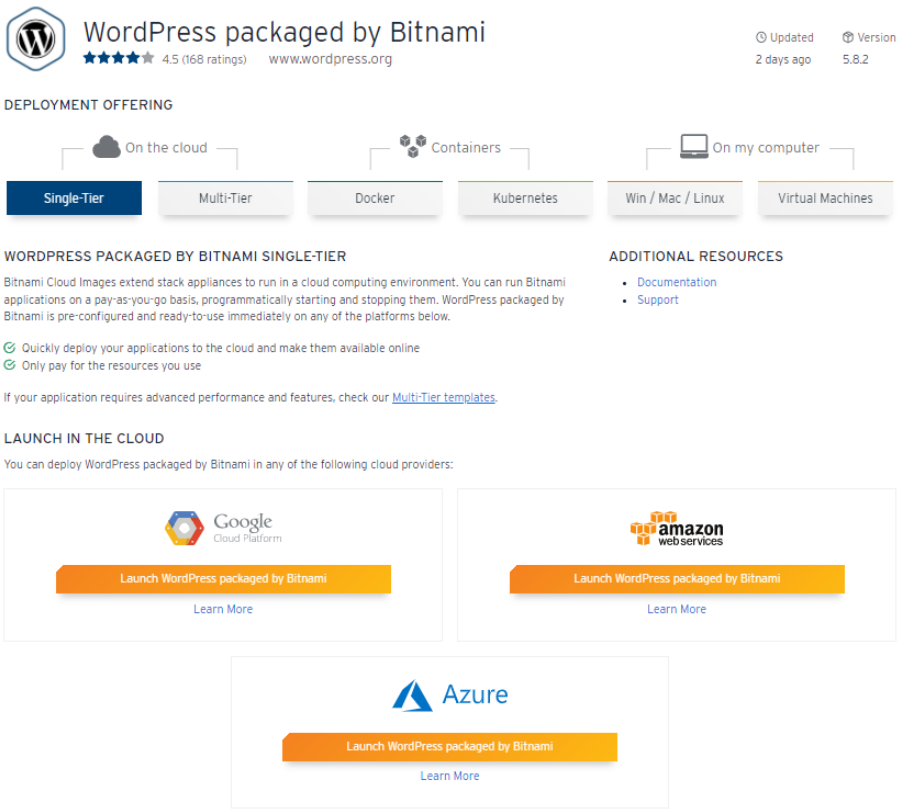


Ilustración 3: WordPress para Intel usando los paquetes de Bitnami

Como se muestra in Ilustración 3, Bitnami y Intel han publicado el WordPress para Intel usando los paquetes de Bitnami en Azure, AWS y GCP. Estos tienen una pila de software preinstalado que es disponibles públicamente para los desarrolladores de software . Para comenzar, va a este enlace: https://bitnami.com/stack/wordpress/cloud. Puede hacer clic el botón de Launch para iniciar la imagen lo que es mejor para su proyecto. Esta sección explica cómo hacer una evaluación comparativa del WordPress que se ha instalado en una máquina virtual en la Azure, AWS o GCP.

Las máquinas virtuales

Todos los ejemplos son usando uno 16 vCPU VM servidor y dos 16 vCPU VM clientes.

	AWS	GCP	Azure
VM servidor	M6i.4xlarge	N2-standard-16 (CPU: Intel IceLake o más tarde)	Standard_D16_v5
2 VMs clientes	M5.4xlarge	N2-standard-16 (CPU: Intel CascadeLake o más tarde)	Standard_D16_v4

4.1. Configuración de la Azure

Inicie una sesión en su cuenta o cree una cuenta. Va a ver la información "Get It Now" (consíguelo ahora) como se muestra en Ilustración 4:

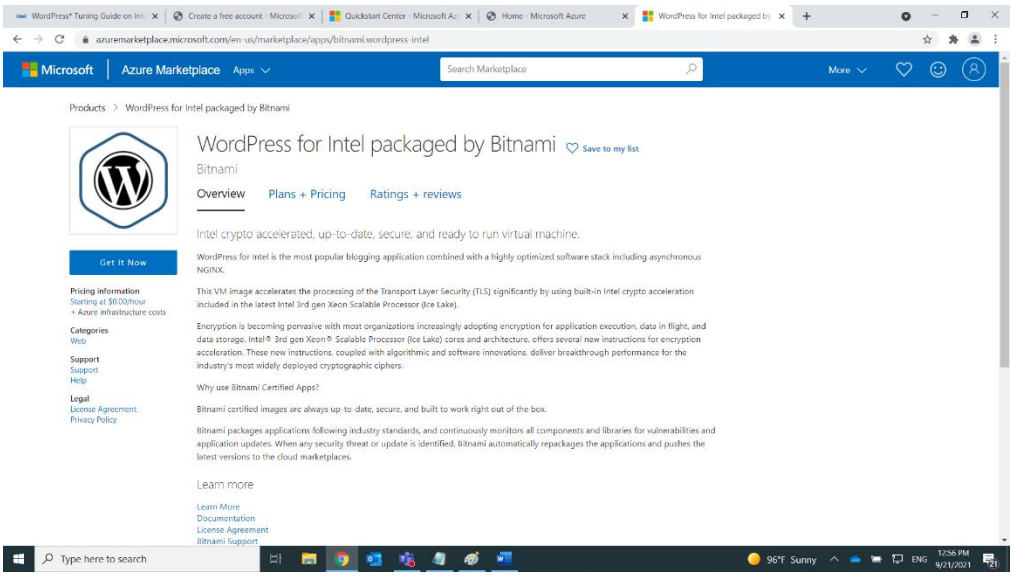


Ilustración 4: Haga clic el botón, Get It Now, para descargar el WordPress para Intel usando los paquetes de Bitnami

Haga clic el botón, **Get It Now,** entonces, haga clic el botón, **Continue.** Elija su cuenta. Va a ver el WordPress para Intel usando los paquetes de Bitnami como se muestra en Ilustración 5.

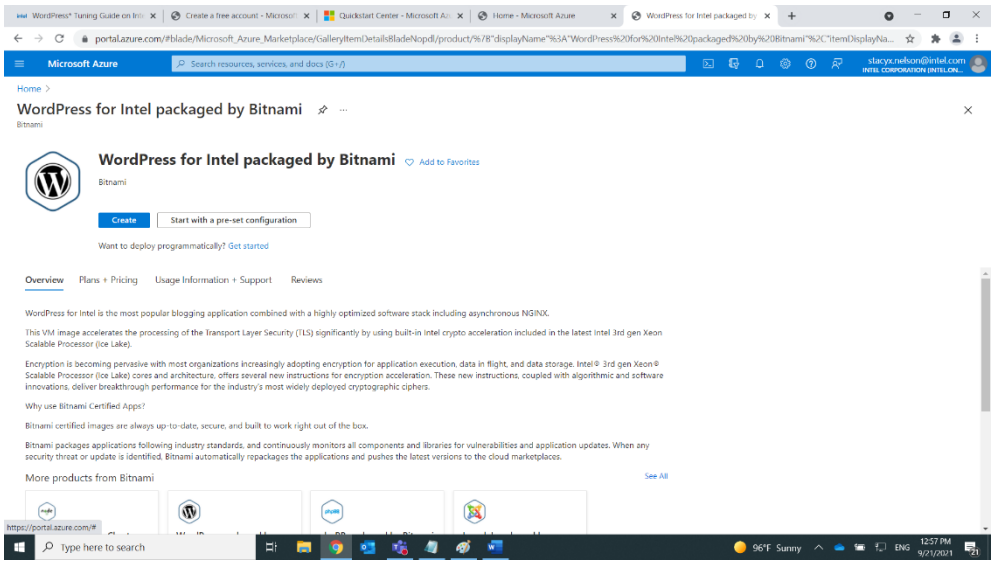


Ilustración 5: Haga clic el botón, Create, para abrir las fichas de entrada de datos para WordPress para Intel usando los paquetes de Bitnami

Haga clic en el botón, Create. Va a ver las fichas de entrada de datos, Create a virtual machine (crear una máquina virtual). La ficha, Basics, va a abrir como se muestra en Ilustración 6.

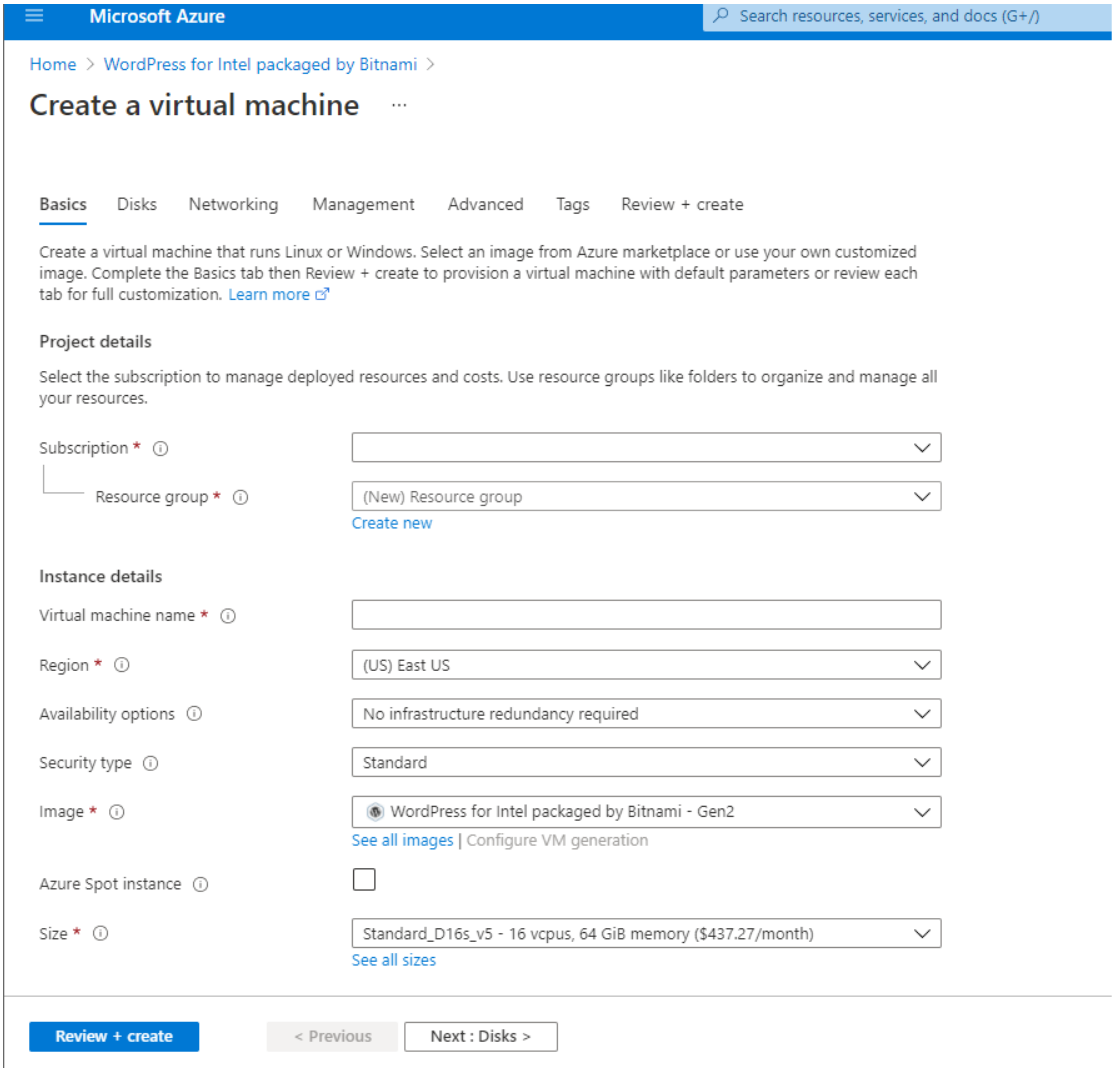


Ilustración 6: La configuración básica

Elija la opción, 3rd Generation Intel® Xeon® Scalable Processor instance (v5 family, default 16 vCPUs). Elija su método de autenticación. Abre la ficha de entrada de datos, Disks (discos), como se muestra en Ilustración 7.

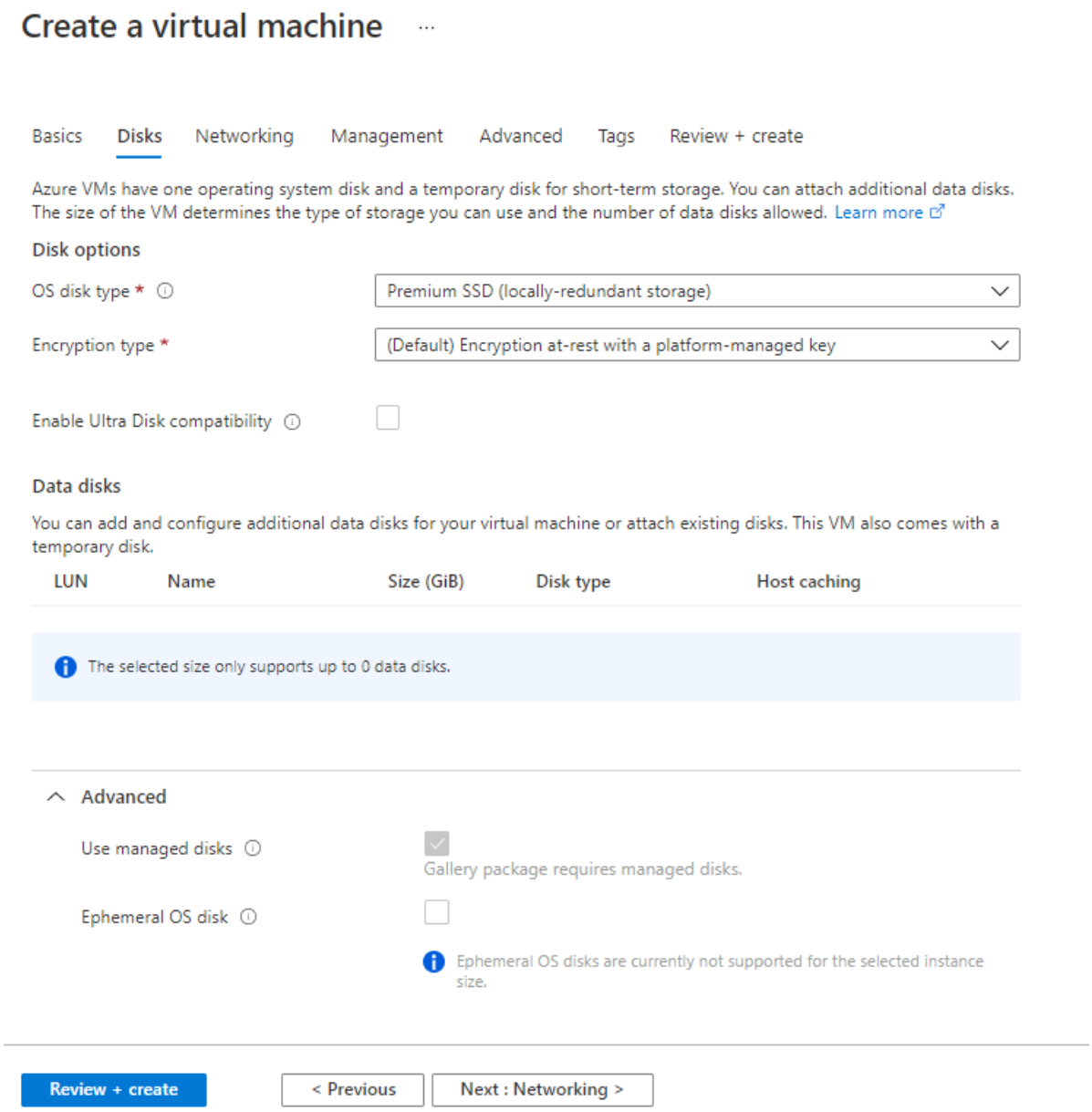


Ilustración 7: Elija la opción, Choose the Disk, y añada más discos duros, si necesario

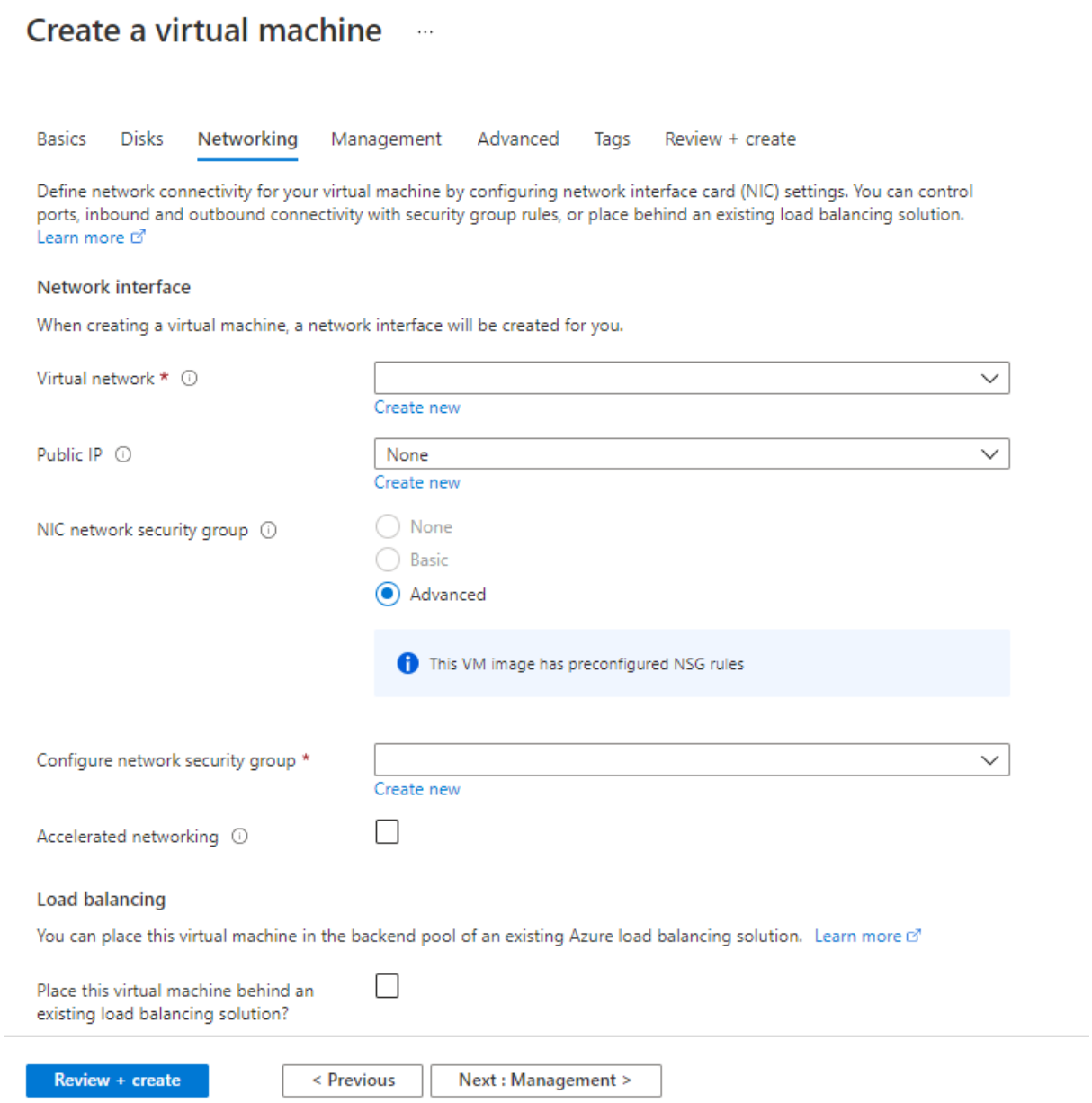


Ilustración 8: Asigne su máquina virtual al red y a solución que se puede equilibrar la carga

Siga las instrucciones para crear dos máquinas virtuales Cascadelake. Los dos deben estar en lo mismo red o clúster privado con su VM servidor. Para su información referencia, Cascadelake 16 core VMs son Standard_D16_v4 (Cascadelake CPU).

Cuando termina la configuración, haga clic el botón, Review + create.

4.2. Configuración de la GCP

Abra la página de web para la plataforma de la nube Google (la sigla en inglés: GCP) como se muestra en Ilustración 9. Haga clic el botón, Launch, para abrir una página de web para entradas los datos para la configuración del WordPress para Intel usando los paquetes de Bitnami.

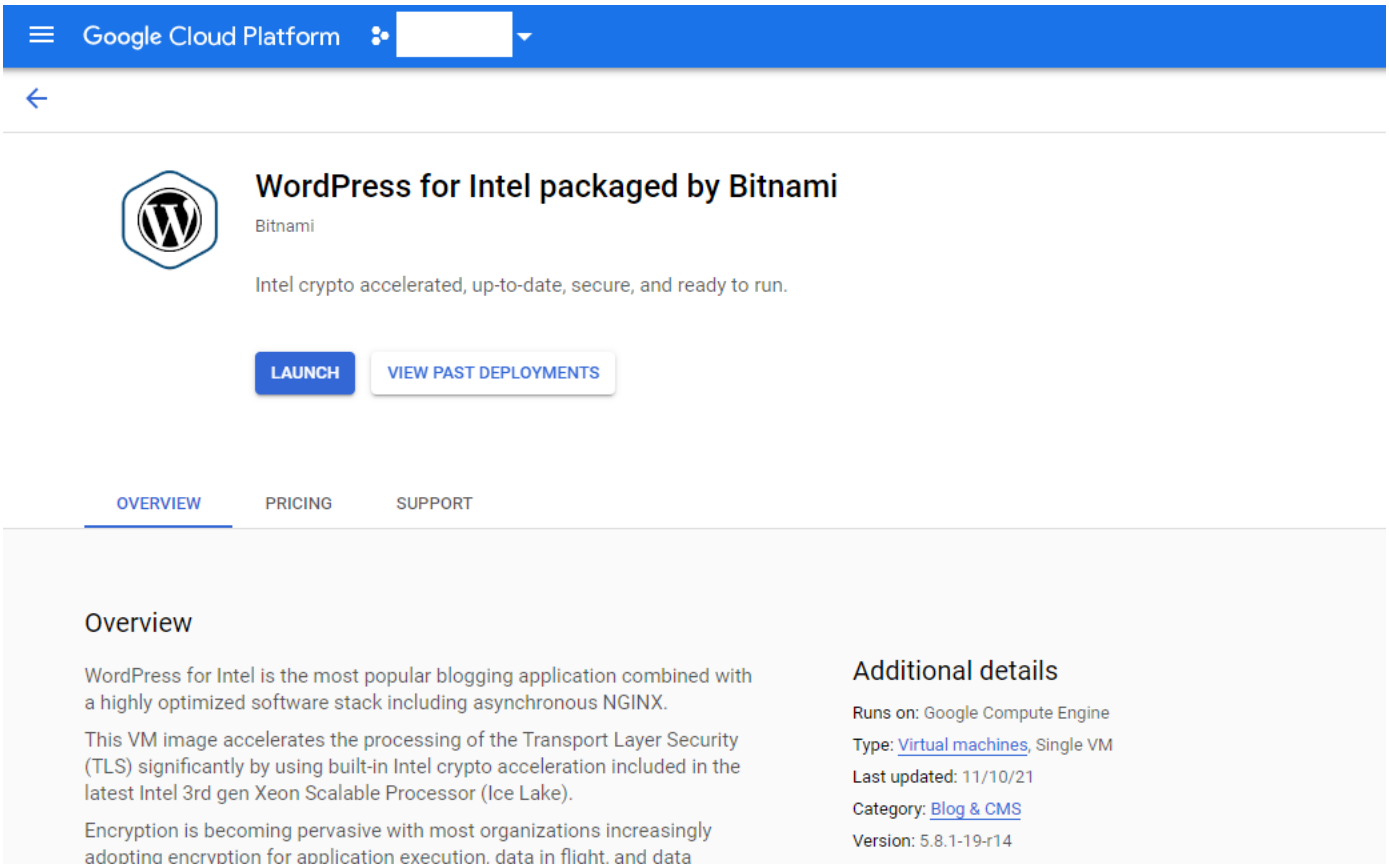
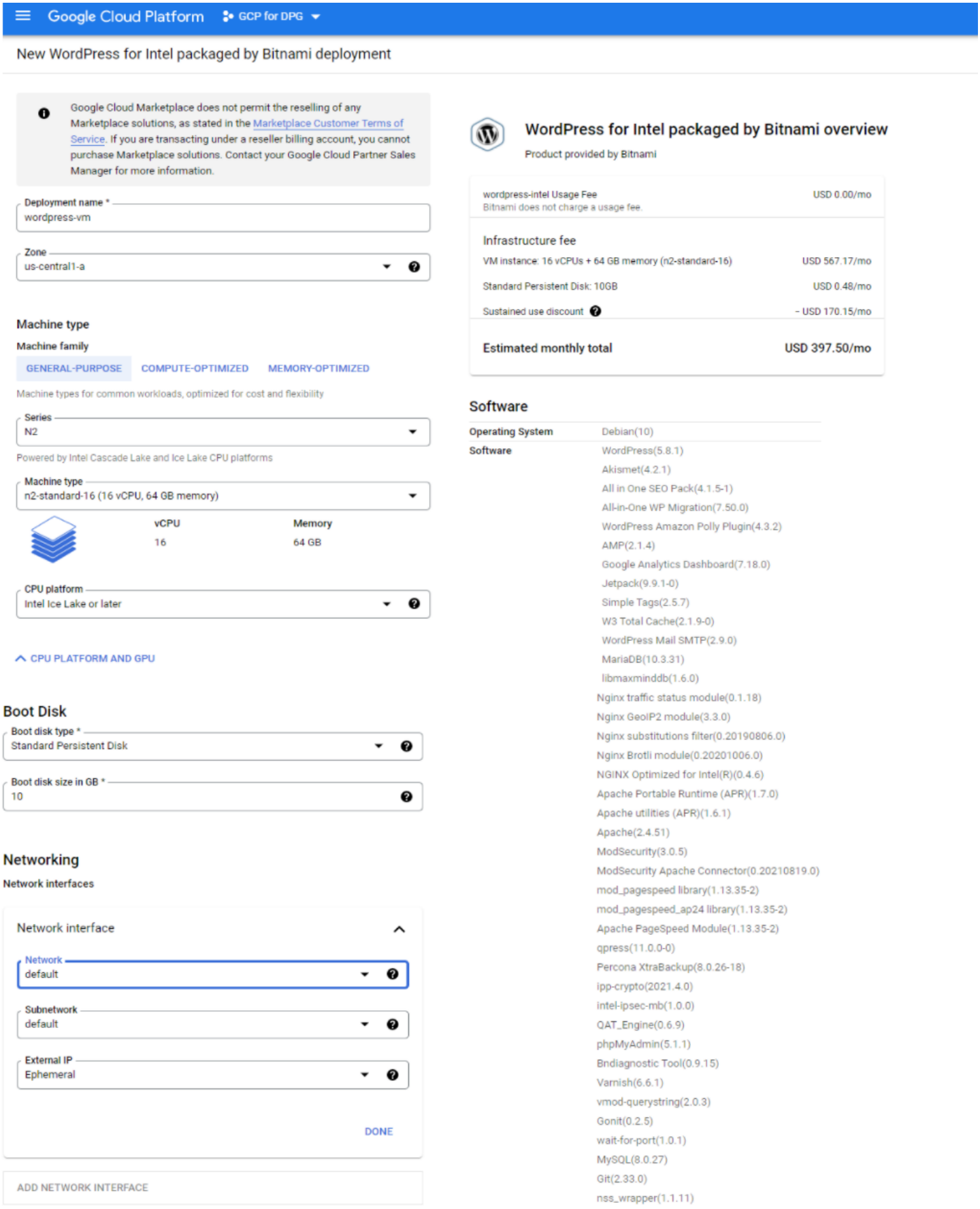


Ilustración 9: Abra WordPress para Intel usando los paquetes de Bitnami en GCP


Google Cloud Platform
GCP for DPG
New WordPress for Intel packaged by Bitnami deployment
Google Cloud Marketplace does not permit the reselling of any Marketplace solutions, as stated in the Marketplace Customer Terms of Service. If you are transacting under a reseller billing account, you cannot purchase Marketplace solutions. Contact your Google Cloud Partner Sales Manager for more information.
Deployment name *
wordpress-vm
Zone
us-central1-a
Machine type
Machine family
GENERAL-PURPOSE
COMPUTE-OPTIMIZED
MEMORY-OPTIMIZED
Machine types for common workloads, optimized for cost and flexibility
Series
N2
Powered by Intel Cascade Lake and Ice Lake CPU platforms
Machine type
n2-standard-16 (16 vCPU, 64 GB memory)
vCPU
16
Memory
64 GB
CPU platform
Intel Ice Lake or later
CPU PLATFORM AND GPU
Boot Disk
Boot disk type *
Standard Persistent Disk
Boot disk size in GB *
10
Networking
Network interfaces
Network interface
Network
default
Subnetwork
default
External IP
Ephemeral
DONE
ADD NETWORK INTERFACE
WordPress for Intel packaged by Bitnami overview
Product provided by Bitnami
wordpress-intel Usage Fee
Bitnami does not charge a usage fee.
USD 0.00/mo
Infrastructure fee
VM instance: 16 vCPUs + 64 GB memory (n2-standard-16)
USD 567.17/mo
Standard Persistent Disk: 10GB
USD 0.48/mo
Sustained use discount
- USD 170.15/mo
Estimated monthly total
USD 397.50/mo
Software
Operating System
Debian(10)
Software
WordPress(5.8.1)
Akismet(4.2.1)
All in One SEO Pack(4.1.5-1)
All-in-One WP Migration(7.50.0)
WordPress Amazon Polly Plugin(4.3.2)
AMP(2.1.4)
Google Analytics Dashboard(7.18.0)
Jetpack(9.9.1-0)
Simple Tags(2.5.7)
W3 Total Cache(2.1.9-0)
WordPress Mail SMTP(2.9.0)
MariaDB(10.3.31)
libmaxminddb(1.6.0)
Nginx traffic status module(0.1.18)
Nginx GeoIP2 module(3.3.0)
Nginx substitutions filter(0.20190806.0)
Nginx Brotli module(0.20201006.0)
NGINX Optimized for Intel(R)(0.4.6)
Apache Portable Runtime (APR)(1.7.0)
Apache utilities (APR)(1.6.1)
Apache(2.4.51)
ModSecurity(3.0.5)
ModSecurity Apache Connector(0.20210819.0)
mod_pagespeed library(1.13.35-2)
mod_pagespeed_ap24 library(1.13.35-2)
Apache PageSpeed Module(1.13.35-2)
qpress(11.0.0-0)
Percona XtraBackup(8.0.26-18)
ipp-crypto(2021.4.0)
intel-ipsec-mb(1.0.0)
QAT_Engine(0.6.9)
phpMyAdmin(5.1.1)
Bndiagnostic Tool(0.9.15)
Varnish(6.6.1)
vmod-querystring(2.0.3)
Gonit(0.2.5)
wait-for-port(1.0.1)
MySQL(8.0.27)
Git(2.33.0)
nss_wrapper(1.1.11)

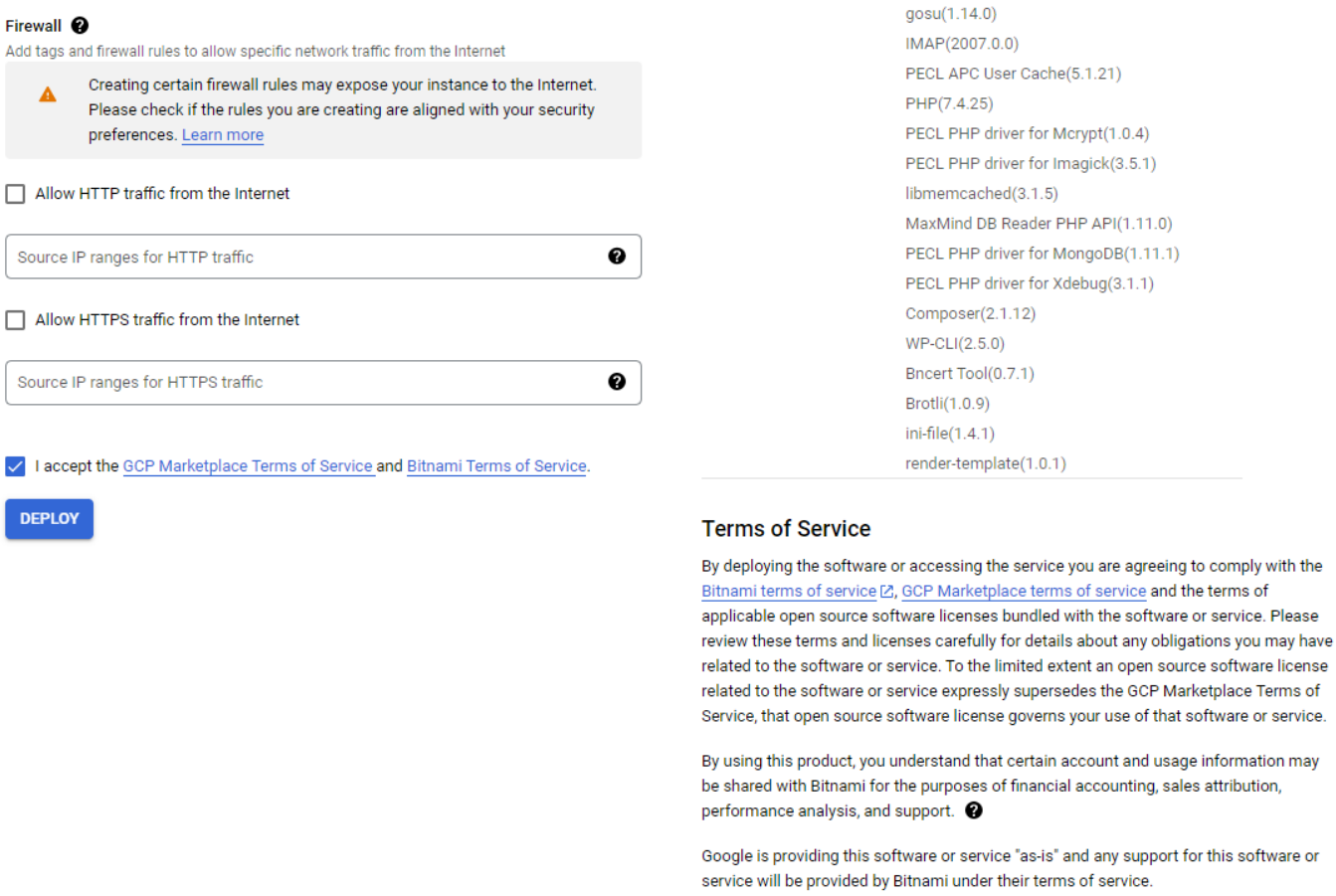


Ilustración 10: La configuración de WordPress para Intel usando los paquetes de Bitnami en GCP

Llena el formulario como se muestra en Ilustración 10 usando su nombre, zona, tipo de máquina y otros detalles del proyecto especifica. Siga estos pasos para crear su red VPC y sus reglas firewall:

1. Desde el menú Navigation, elija VPC network -> VPC networks
2. Cree una red VPC con una subred con sus direcciones IP especificado, por ejemplo: 10.0.0.0/20 en lo mismo región con su VM
3. Desde el menú Navigation, elija VPC network -> Firewall. Cree las reglas firewall de acuerdo con su directiva de red. Se recomendamos permitir que los puertos 22 (SSH), 80 (HTTP) y 443 (HTTPS) se conecten por su la red de su máquina virtual.

Después de implementa su VM, desde el menú Navigation, elija Compute Engine -> VM instances -> Select wordpress-vm-vm. Ahora puede acceso usando su llave SSH. Si no tiene una llave SSH, edita la máquina virtual. Desde el menú elija SSH Keys -> Show and edit. Copie la llave pública SSH para su nombre de usuario y haga clic el botón Save.

Se da cuento que el IP pública y el IP privado primario en la sección: wordpress-vm-vm" in the "Network interfaces. Ahora, puede SSH a su instancia usando su método preferido, por ejemplo: ssh username@<External IP>.

Siga estas instrucciones para crear dos máquinas virtuales Cascadelake. Es necesario que los dos son en lo mismo red o clúster privado con su máquina virtual servidor.

4.3. Configuración de la AWS

Va a AWS y abra la página de web, Create una VPC como se muestra en Ilustración 11. Cree un VPC y una subred de bajo el VPC en lo mismo región con su AMI.

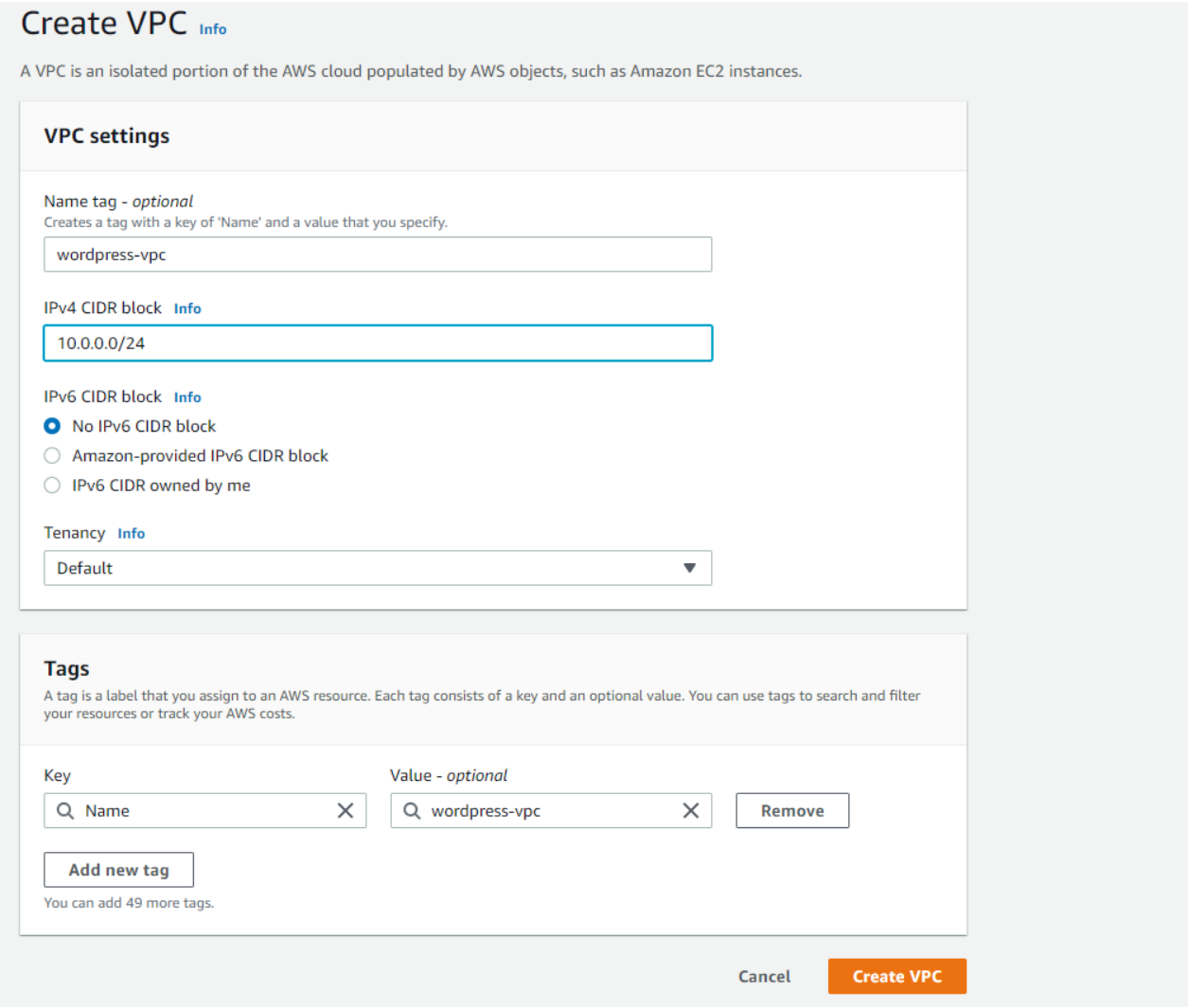
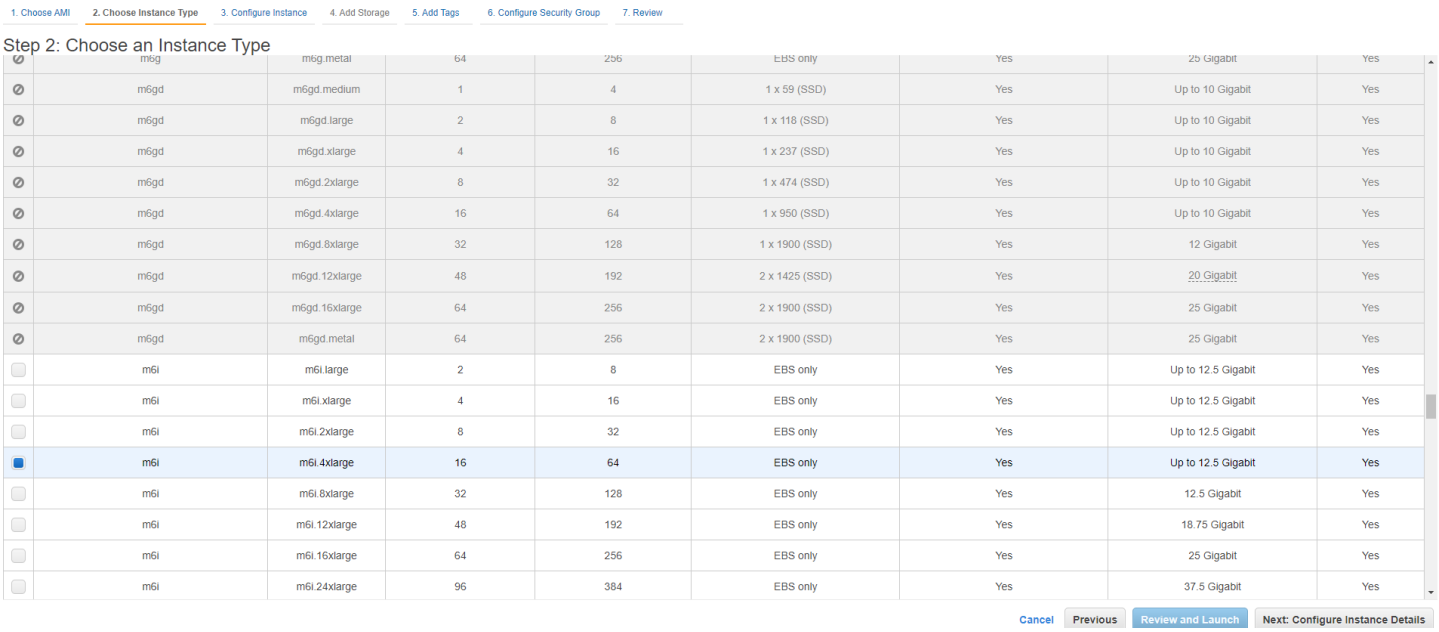


Ilustración 11: Cree un VPC en AWS

Cree una subred de bajo lo mismo nombre de VPC en lo mismo región con su AMI. En el grupo de seguridad, cree las reglas firewall se acuerda su directiva de red. Se recomendamos permitir que los puertos 22 (SSH), 80 (HTTP) y 443 (HTTPS) se conecten por su red a su máquina virtual.

Abra el asistente para instancia usando: AWS Home->Services: Choose EC2->Launch instance wizard. Puede elegir un tipo de instancia como se muestra en Ilustración 12.



m6g	m6g.metal	64	256	EBS only	Yes	25 Gigabit	Yes
m6gd	m6gd.medium	1	4	1 x 59 (SSD)	Yes	Up to 10 Gigabit	Yes
m6gd	m6gd.large	2	8	1 x 118 (SSD)	Yes	Up to 10 Gigabit	Yes
m6gd	m6gd.xlarge	4	16	1 x 237 (SSD)	Yes	Up to 10 Gigabit	Yes
m6gd	m6gd.2xlarge	8	32	1 x 474 (SSD)	Yes	Up to 10 Gigabit	Yes
m6gd	m6gd.4xlarge	16	64	1 x 950 (SSD)	Yes	Up to 10 Gigabit	Yes
m6gd	m6gd.8xlarge	32	128	1 x 1900 (SSD)	Yes	12 Gigabit	Yes
m6gd	m6gd.12xlarge	48	192	2 x 1425 (SSD)	Yes	20 Gigabit	Yes
m6gd	m6gd.16xlarge	64	256	2 x 1900 (SSD)	Yes	25 Gigabit	Yes
m6gd	m6gd.metal	64	256	2 x 1900 (SSD)	Yes	25 Gigabit	Yes
m6i	m6i.large	2	8	EBS only	Yes	Up to 12.5 Gigabit	Yes
m6i	m6i.xlarge	4	16	EBS only	Yes	Up to 12.5 Gigabit	Yes
m6i	m6i.2xlarge	8	32	EBS only	Yes	Up to 12.5 Gigabit	Yes
m6i	m6i.4xlarge	16	64	EBS only	Yes	Up to 12.5 Gigabit	Yes
m6i	m6i.8xlarge	32	128	EBS only	Yes	12.5 Gigabit	Yes
m6i	m6i.12xlarge	48	192	EBS only	Yes	18.75 Gigabit	Yes
m6i	m6i.16xlarge	64	256	EBS only	Yes	25 Gigabit	Yes
m6i	m6i.24xlarge	96	384	EBS only	Yes	37.5 Gigabit	Yes

Ilustración 12: Elija un tipo de instancia

Elija su tipo de instancia. En este ejemplo, elegimos m6i.4xlarge instance. Configure sus detalles como se muestra en Ilustración 13.

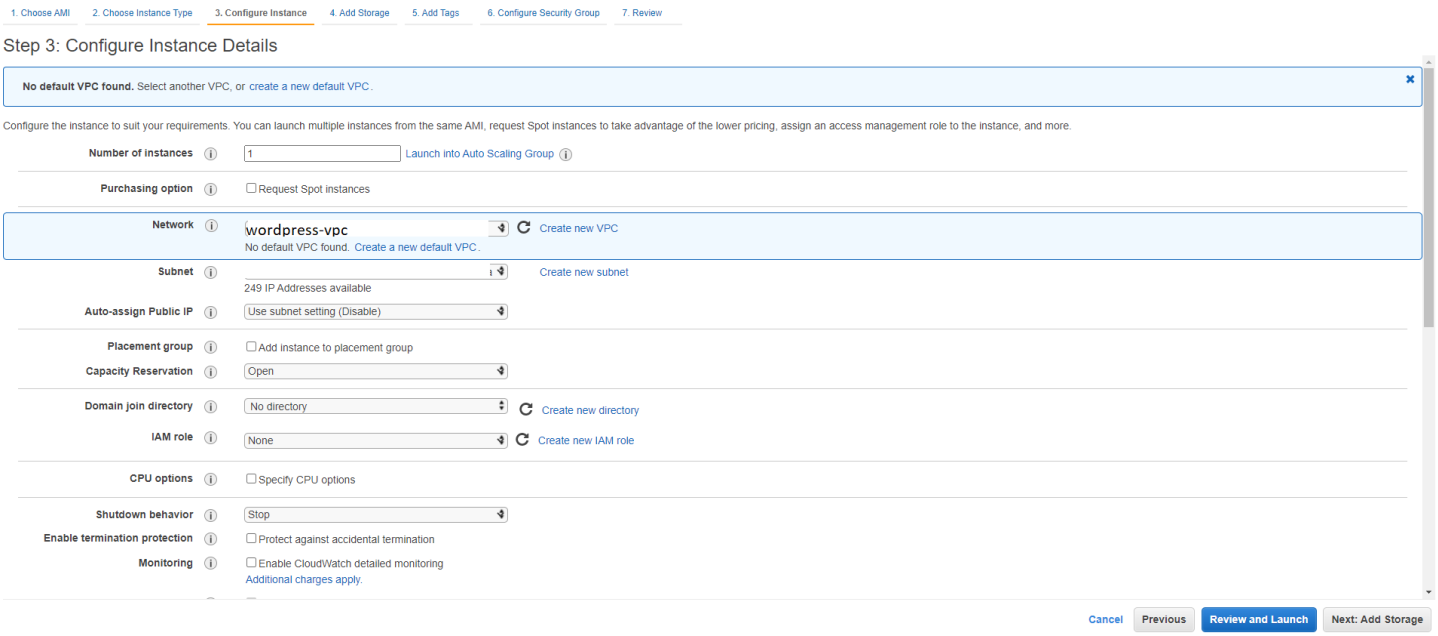


Ilustración 13: Configure las detalles del instancia

Configure otros ajustes según las necesidades de su proyecto. Siga estas instrucciones para crear dos Cascadelake VM instancias. Tiene que estar en lo mismo red o clúster con su VM servidor.

4.4. La pila de software

Las imágenes de WordPress para Intel usando los paquetes de Bitnami tienen todas pilas de software preinstalado incluyendo

los librarías cifrados con optimizaciones Intel. Lugares importantes de estos librarías y carpetas con ajustes se muestran de bajo:

Libraries cifrados:

/opt/bitnami/common/lib/
/opt/bitnami/common/lib/engines-1.1/

PHP:

/opt/bitnami/php

Nginx:

/opt/bitnami/nginx

Las configuraciones Nginx Tunable son queda aquí:

/opt/bitnami/nginx/conf/nginx.conf
/opt/bitnami/nginx/conf/server_blocks/wordpress-https-server-block.conf

5. Configuración de evaluación comparativa

Apunte: Esta sección es para imágenes en servidores físicos y imágenes en servidores virtuales.

La base de datos para WordPress:

Para probar que el PHP está funciona bien, recomendamos pobla su base de datos con varios tipos de comentarios de WordPress. Una base de datos ejemple está aquí:

https://github.com/intel/Updates-for-OSS-Performance/blob/main/targets/wordpress/dbdump_v5.sql.gz

Si usa esta base de datos, por favor cambiar puerta 8085 con su puerto HTTPS (Se usa 443 en el WordPress para Intel usando los paquetes de Bitnami.)

Instala el cliente, Siege, en dos VM clientes Azure con procesadores escalables Intel® Xeon® de 2 a generación o dos VM clientes con procesadores escalables Intel® Xeon® de 2 a generación, depende de cual plataforma está usando. Para configurar cada máquina virtual, SSH a cada máquina virtual cliente y siga las pasos de bajo:

Instale el Siege:

```
sudo apt update
sudo apt install libssl-dev #install ssl headers
wget http://download.joedog.org/siege/siege-2.78.tar.gz
gunzip siege-2.78.tar.gz
tar -xvf siege-2.78.tar
cd siege-2.78
./configure --prefix=/usr/local --with-ssl=/usr/bin/openssl
make clean
make
make install
wget https://github.com/intel/Updates-for-OSS-Performance/raw/main/targets/wordpress/WordpressTarget_v5.urls
sed -i 's/__HTTP_HOST__/10.0.0.1/g' WordpressTarget_v5.urls, where 10.0.0.1 is an example internal address for the ICE LAKEserver VM. Please replace it accordingly.
 sed -i 's/__HTTP_PORT__/443/g' WordpressTarget_v5.urls
 sed -i 's/http/https/g' WordpressTarget_v5.urls
From both the siege VMs, launch the client requests simultaneously:

siege -f ./WordpressTarget_v5.urls -c 225 --time=60S, where 225 is the concurrency and time is the benchmarking time in seconds.
```

Recomendamos tiene por lo menos uno minuto de evaluación comparativa. El ajuste, concurrency (la simultaneidad), se ha determinado para lograr 90+% utilización CPU en el servidor ICE LAKEVM y ha presentado 0 transacciones fallidas en el Siege. Ejecute esta evaluación comparativa por 6 veces. La primera vez es una preparación, así que la descarte. Usa la mediana de las 5 otras ejecuciones. La velocidad de transmisión total es la tasa de transacción combinada de ambos clientes. No hemos visto ninguna desviación estándar importante (>0.5%) entre las ejecuciones. Observe los resultados. Cambie las opciones concurrency hasta que la utilización de la CPU en la máquina virtual servidor es >90% (como se indica a "sar 1"). Tenga en cuenta cualquier anomalía. Modifique las opciones concurrency hasta que la utilización de la CPU en la máquina virtual del servidor sea >90% (como se indica de "sar 1"). En lo mismo momento asegúrese que las "transacciones fallidas" permanezcan 0.

6. Prácticas recomendadas para pruebas y verificación

Las recomendaciones in esta guía son solas referencias y ajustes deben ser adoptar después de un repasar por un profesional quien sabe la carga de trabajo WordPress y ajustes sistemas.

- Se espera ver ~90+% utilización de CPU. Use herramientas como sar-htop para verificar su utilización de CPU
- Ejecute a lo menos que 7 ejecuciones para asegurar que la desviación estándar está dentro del 5%
- Reinicie el servicio MariaDB después de cada ejecución para vaciar query_cache. Esta es una recomendación especifica de la carga de trabajo y no es una recomendación general

7. Más Referencias

http://www.brianhoshi.com/blog/how-to-set-up-wordpress-on-aws-ec2-with-bitnamis-ami/

Notices & Disclaimers

Intel technologies may require enabled hardware, software or service activation.

No product or component can be absolutely secure.

Your costs and results may vary.

Code names are used by Intel to identify products, technologies, or services that are in development and not publicly available. These are not "commercial" names and not intended to function as trademarks

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request